

Microsoft Fabric Warehouse

Transactions & Isolation Levels

DBA Technical Reference | Volume 5 of 9

April 2025 | Version 1.0

Table of Contents

1.	Transaction Support Overview	3
2.	Snapshot Isolation Explained	3
3.	BEGIN / COMMIT / ROLLBACK	4
4.	Multi-Table Transactions	4
5.	Lock Behavior in Fabric Warehouse	5
6.	Write-Write Conflicts	5
7.	Implicit vs Explicit Transactions	6
8.	Transaction Monitoring with DMVs	6
9.	Error Handling in Transactions	7
10.	ETL Design Patterns for Transactions	8

1. Transaction Support Overview

Fabric Warehouse supports full ACID transactions — Atomicity, Consistency, Isolation, and Durability. This is built on the Delta Lake transaction log, which records all changes to data as ordered, atomic commits.

ACID Property	Fabric Warehouse Implementation
Atomicity	All statements in a transaction succeed or all are rolled back. Delta log ensures atomic commits.
Consistency	Constraints (NOT NULL, PRIMARY KEY, UNIQUE) are enforced. Schema is validated at write time.
Isolation	Snapshot Isolation only — readers see a consistent snapshot; not blocked by writers.
Durability	Committed transactions are written to Delta Lake log and Parquet files on OneLake (geo-redundant storage).

INFO: Fabric Warehouse does NOT support READ COMMITTED, REPEATABLE READ, or SERIALIZABLE isolation levels. Snapshot Isolation is the only available isolation level and cannot be changed.

2. Snapshot Isolation Explained

Under Snapshot Isolation, every transaction reads from a consistent point-in-time snapshot of the database. Readers never block writers and writers never block readers.

Scenario	Behavior under Snapshot Isolation
Reader while writer is active	Reader sees the data as it was BEFORE the writer's transaction started
Two concurrent readers	Both see the same snapshot — no conflicts
Writer while reader is active	Writer proceeds without waiting for the reader to finish
Two concurrent writers on same table	Second writer to commit wins; first writer fails with conflict error
Writer and background compaction	Compaction aborts if it detects a conflicting user write (post Oct 2025)

3. BEGIN / COMMIT / ROLLBACK

>> Basic Transaction Structure

```
BEGIN TRANSACTION;

-- Step 1: Insert new records
INSERT INTO dbo.FactSales (OrderDate, CustomerKey, SalesAmount)
VALUES ('2024-06-15', 1001, 2500.00);

-- Step 2: Update summary table
UPDATE dbo.SalesSummary
SET TotalAmount = TotalAmount + 2500.00
WHERE SummaryDate = '2024-06-15';

COMMIT TRANSACTION;
```

>> Transaction with Error Handling

```
BEGIN TRANSACTION;

BEGIN TRY

UPDATE dbo.FactSales
SET SalesAmount = SalesAmount * 1.10
WHERE OrderDate = '2024-06-15';

UPDATE dbo.SalesSummary
SET TotalAmount = TotalAmount * 1.10
WHERE SummaryDate = '2024-06-15';

COMMIT TRANSACTION;

END TRY

BEGIN CATCH

ROLLBACK TRANSACTION;

THROW; -- re-raise the error to the calling application

END CATCH;
```

4. Multi-Table Transactions

Fabric Warehouse supports transactions spanning multiple tables. This is essential for maintaining referential integrity and data consistency in star schema ETL pipelines.

>> Multi-Table ETL Transaction Pattern

```
BEGIN TRANSACTION;

BEGIN TRY

  -- Load dimension first
  INSERT INTO dbo.DimProduct (ProductKey, ProductName, Category)
  SELECT s.ProductKey, s.ProductName, s.Category
  FROM dbo.StagingProduct s
  WHERE NOT EXISTS (
    SELECT 1 FROM dbo.DimProduct d
    WHERE d.ProductKey = s.ProductKey
  );

  -- Then load fact (FK relationship maintained atomically)
  INSERT INTO dbo.FactSales (OrderDate, ProductKey, SalesAmount)
  SELECT OrderDate, ProductKey, SalesAmount
  FROM dbo.StagingFactSales
  WHERE LoadBatch = CAST(GETUTCDATE() AS DATE);

  -- Clear staging
  DELETE FROM dbo.StagingFactSales
  WHERE LoadBatch = CAST(GETUTCDATE() AS DATE);

  COMMIT TRANSACTION;

END TRY

BEGIN CATCH

  ROLLBACK TRANSACTION;

  THROW;

END CATCH;
```

5. Lock Behavior in Fabric Warehouse

Lock management in Fabric Warehouse is table-level, not row-level. This is fundamentally different from SQL Server, where row-level locking is the default for DML operations.

Operation	Lock Acquired	Duration
SELECT	No lock — reads from snapshot	N/A — non-blocking
INSERT	Table-level write lock	For duration of INSERT statement
UPDATE	Table-level write lock	For duration of UPDATE statement
DELETE	Table-level write lock	For duration of DELETE statement
MERGE	Table-level write lock	For duration of MERGE statement
DDL (CREATE/ALTER/DROP)	Schema lock	For duration of DDL statement

INFO: INSERT operations rarely cause conflicts because they write to new Parquet files. The Delta log is updated atomically at commit time. Only UPDATES and DELETES modify existing data versions and are prone to conflicts.

6. Write-Write Conflicts

A write-write conflict occurs when two concurrent transactions attempt to modify the same table. Under Snapshot Isolation, the transaction that commits second will fail with a conflict error.

>> Write-Write Conflict Scenario

```
-- Session 1: Starts UPDATE at 10:00:00
BEGIN TRANSACTION;
UPDATE dbo.FactSales SET SalesAmount = SalesAmount * 1.05
WHERE Region = 'North';
-- Session 1 is still running...

-- Session 2: Starts UPDATE at 10:00:05 -- overlaps with Session 1
BEGIN TRANSACTION;
UPDATE dbo.FactSales SET SalesAmount = SalesAmount * 0.90
WHERE Region = 'North';
COMMIT; -- Session 2 commits at 10:00:10

-- Session 1 tries to commit at 10:00:15 -- FAILS with conflict error
-- Because Session 2 already modified the Delta log for this table
COMMIT; -- Error: Write-write conflict detected
```

>> Retry Pattern for Write-Write Conflicts

```
DECLARE @retry INT = 0;
DECLARE @max_retries INT = 3;

WHILE @retry < @max_retries
BEGIN
    BEGIN TRY
        BEGIN TRANSACTION;

        UPDATE dbo.FactSales
        SET SalesAmount = SalesAmount * 1.05
        WHERE Region = 'North';

        COMMIT TRANSACTION;
        SET @retry = @max_retries; -- Exit loop on success
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK;
        SET @retry = @retry + 1;
        IF @retry = @max_retries THROW; -- Re-raise after max retries
        WAITFOR DELAY '00:00:02'; -- Wait 2 seconds before retry
    END CATCH
END;
```

7. Implicit vs Explicit Transactions

Type	Behavior	Best Practice
Implicit (default)	Each statement is its own auto-committed transaction	Safe for single statements; no BEGIN/COMMIT needed
Explicit	Developer wraps statements in BEGIN TRANSACTION...COMMIT	Required for multi-statement atomicity
Explicit (long-running)	Transaction held open for extended period	Avoid — increases conflict risk with compaction

WARNING: Avoid holding explicit transactions open for long periods (seconds to minutes). Long-running open transactions increase the probability of write-write conflicts with both concurrent users and the background compaction service.

8. Transaction Monitoring with DMVs

>> View All Active Transactions

```
SELECT
  t.transaction_id,
  t.name AS transaction_name,
  t.transaction_begin_time,
  DATEDIFF(SECOND,
    t.transaction_begin_time,
    GETUTCDATE()) AS age_seconds,
  t.transaction_type,
  t.transaction_state
FROM sys.dm_tran_active_transactions t
ORDER BY t.transaction_begin_time;
```

>> Find Long-Running Open Transactions (Over 60 Seconds)

```
SELECT
  st.session_id,
  s.login_name,
  t.transaction_begin_time,
  DATEDIFF(SECOND,
    t.transaction_begin_time,
    GETUTCDATE()) AS age_seconds
FROM sys.dm_tran_session_transactions st
JOIN sys.dm_exec_sessions s ON st.session_id = s.session_id
JOIN sys.dm_tran_active_transactions t ON st.transaction_id = t.transaction_id
WHERE DATEDIFF(SECOND, t.transaction_begin_time, GETUTCDATE()) > 60
ORDER BY age_seconds DESC;
```

9. Error Handling in Transactions

>> Stored Procedure with Full Error Handling

```
CREATE PROCEDURE dbo.usp_LoadDailySales
@LoadDate DATE
AS
BEGIN
SET NOCOUNT ON;

BEGIN TRANSACTION;

BEGIN TRY

-- Validate staging data exists
IF NOT EXISTS (
SELECT 1 FROM dbo.StagingFactSales
WHERE LoadDate = @LoadDate
)
BEGIN
RAISERROR('No staging data found for date: %s', 16, 1,
@LoadDate);
END;

-- Load fact table
INSERT INTO dbo.FactSales (OrderDate, CustomerKey, SalesAmount)
SELECT OrderDate, CustomerKey, SalesAmount
FROM dbo.StagingFactSales
WHERE LoadDate = @LoadDate;

-- Remove loaded staging rows
DELETE FROM dbo.StagingFactSales WHERE LoadDate = @LoadDate;

COMMIT TRANSACTION;

END TRY
BEGIN CATCH
IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;

DECLARE @msg NVARCHAR(4000) = ERROR_MESSAGE();
DECLARE @sev INT = ERROR_SEVERITY();
DECLARE @st INT = ERROR_STATE();
RAISERROR(@msg, @sev, @st);
END CATCH;

END;
```

10. ETL Design Patterns for Transactions

Pattern	Description	When to Use
Staging Table + Atomic Load	Load to staging first, then INSERT...SELECT into target in one transaction	Large daily/incremental batch loads
Upsert with MERGE	MERGE staging into target — handles insert and update in one statement	SCD Type 1 dimension updates
Delete + Reload	DELETE target partition rows, reload from staging atomically	Full partition refresh pattern
Idempotent Loads	Use WHERE NOT EXISTS or MERGE to prevent duplicate rows	Pipelines that may re-run on failure
Two-Phase Commit	Load dimensions first (commit), then load facts	Maintaining FK relationships across loads

>> Upsert Pattern Using MERGE

```

MERGE dbo.DimCustomer AS target
USING dbo.StagingCustomer AS source
ON target.CustomerKey = source.CustomerKey
WHEN MATCHED AND (
    target.CustomerName <> source.CustomerName OR
    target.Email <> source.Email
) THEN
UPDATE SET
    target.CustomerName = source.CustomerName,
    target.Email = source.Email,
    target.UpdatedDate = GETUTCDATE()
WHEN NOT MATCHED BY TARGET THEN
INSERT (CustomerKey, CustomerName, Email, CreatedDate)
VALUES (source.CustomerKey, source.CustomerName,
    source.Email, GETUTCDATE());

```